Optimal Repairs for Unary Functional Dependencies: Resolving the Case of Updates

Benny Kimelfeld
Technion – Israel Institute of Technology

Ester Livshits
Technion – Israel Institute of Technology

Abstract

If a table violates its required set of functional dependencies (FDs), what is the minimum number of cell changes needed to restore consistency? This fundamental problem, known as finding an optimal update repair (U-repair), is known to admit polynomial-time algorithms only for a small number of specific FD sets. Whether additional tractable cases exist has remained open. The only established hardness result for this problem is due to Kolahi and Lakshmanan (2009); subsequent attempts to prove hardness for additional cases have failed, leaving these cases unresolved. In this work, we make substantial progress on this open problem by completely resolving the case of unary FDs, in which every FD has a single attribute on its left-hand side. We show that every set of unary FDs either falls into one of the previously known tractable classes or makes the problem of finding an optimal U-repair NP-hard.

2012 ACM Subject Classification Information systems → Data management systems

Keywords and phrases Update repairs, functional dependencies

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

1 Introduction

Suppose that a database violates a set of functional dependencies (FDs). *How many cell values need to be changed, at minimum, to restore consistency?* This question defines the problem of computing an *optimal update repair* (or *optimal U-repair*): we seek a database that satisfies the FDs and differs from the original database in as few cells as possible. Despite the elementary formulation of the problem and the central role of FDs in databases, its computational complexity is surprisingly poorly understood. As we explain later, polynomial-time algorithms are known only for a few restricted classes of FD sets, and the only established hardness result applies to a specific configuration of two FDs. Whether further tractable cases exist has remained open.

This problem is a natural instance of the general framework of database repairs. Inconsistencies arise in databases for many reasons. Data may originate from noisy or imprecise sources, such as sensors, social platforms, or automatically extracted content, and may be produced by error-prone processes such as machine learning and generative Artificial Intelligence (AI). Inconsistencies can also emerge when integrating databases from different sources, which may provide conflicting information or represent the same information in incompatible ways. Arenas et al. [2] introduced a principled approach to managing such inconsistencies through the notions of *repairs* and *consistent query answering*. Given a database D that violates a collection of integrity constraints, a repair is a consistent database D' obtained from D through a minimal collection of editing operations. Consistent query answering then seeks the query answers that hold across all repairs.

The repair framework admits many variants, according to three basic choices: the *integrity constraints* that define consistency, the *operations* allowed for restoring consistency, and the criterion used to measure minimality [1]. Integrity constraints commonly include denial

constraints [9], and in particular the classical functional dependencies, as well as inclusion dependencies [5], which include referential constraints. Repair operations may consist of deleting or inserting tuples, or modifying individual cell values. Finally, minimality may be *local*, requiring that no proper subset of the performed operations suffices, or *global*, requiring minimum total cost among all ways of restoring consistency. For instance, when repairs are restricted to tuple deletions, a *subset repair* [6] is locally minimal, whereas a *cardinality repair* [17] minimizes the number of deleted tuples. Operations may also be assigned nonuniform costs, for example to reflect different levels of confidence in data items [10, 13, 17] or distance between the original and the updated values [12, 14].

We study the natural combination in which consistency is defined by FDs, repairs are obtained exclusively through value updates, and minimality is global. In the basic setting, changing a single cell has unit cost, so the objective is precisely to minimize the number of changed cells. Our results extend directly to weighted updates, where cells may have different costs. This notion was called an "optimum V-repair" by Kolahi and Lakshmanan [13], and we adapt the term "optimal U-repair" by Livshits et al. [16].

The current understanding of this optimization problem leaves a substantial gap between tractability and hardness. Livshits et al. [16] established polynomial-time algorithms for two general forms of FD sets, up to equivalence. The first is an *lhs-chain*, where the left-hand sides of the FDs form a chain under inclusion. The second is a *matching constraint* $\{A \to B, B \to A\}$, which we denote by $A \leftrightarrow B$. When all FDs are unary, these results provide tractability essentially only when the FD set reduces to a single determinant $\{A \to X\}$ or to a matching constraint $A \leftrightarrow B$. In addition, when an FD set is a union of FD sets that share no common attributes, then the two corresponding repair problems can be solved separately (hence, $\{A \leftrightarrow B, C \leftrightarrow D\}$ is also tractable).

The known hardness landscape is even more limited. Kolahi and Lakshmanan [13] proved NP-hardness for $\{A \to B, B \to C\}$. Attempts to extend hardness beyond this case have not succeeded. In particular, in their conference version, Livshits et al. [15] claimed hardness for $\{A \leftrightarrow B \to C\}$; an error in the proof was subsequently discovered, as they discuss in the journal version [16]. More recently, Miao et al. [18] claimed NP-hardness for $\{A \to C, B \to C\}$. As we explain in Section 5.1, the proposed reduction does not establish this claim.

Hence, even after restricting attention to *unary FDs* (i.e., FDs with a single attribute on the left-hand side), a basic question has remained unanswered: are the known polynomial-time cases isolated exceptions, or are there additional classes of unary FDs that admit efficient optimal U-repair algorithms? We answer this question completely. We prove a dichotomy for unary FDs: for every set Δ of unary FDs, either Δ falls, up to equivalence, within one of the previously known tractable cases, or computing an optimal U-repair with respect to Δ is NP-hard. In particular, the previously known algorithms exhaust the tractable cases among unary FDs.

2 Preliminaries

We begin with preliminary definitions and notation.

Relations. Throughout the paper, we will assume countably infinite sets **Val** of *values* and **Att** of *attributes*. A *relation schema* is a set $R = \{A_1, \ldots, A_k\}$ of attributes. An *R-tuple* is a function $t : R \to \mathbf{Val}$ that maps each attribute $A \in R$ to a value that we denote by $t[A]$. A *relation* r is associated with a relation schema, denoted $\mathbf{Att}(r)$, a finite set of tuple

identifiers, denoted $\mathsf{tids}(r)$, and a mapping from $\mathsf{tids}(r)$ to $\mathbf{Att}(r)$-tuples. We say that r is a relation *over* the relation schema $\mathbf{Att}(r)$. We denote by $r[i]$ the tuple that r maps to the identifier i. Hence, $r[i][A]$ is the value that tuple i has for the attribute A.

▶ Remark 1. Note that we allow duplicate tuples, since we do not assume that tuples with different identifiers are necessarily distinct. ◁

Let X be a set of attributes. We denote by $\pi_X r$ the projection of r onto X. Formally, $\pi_X r$ is the relation r' such that $\mathbf{Att}(r') = X$, $\mathsf{tids}(r') = \mathsf{tids}(r)$, and $r'[i][A] = r[i][A]$ for every $A \in \mathbf{Att}(r) \cap X$. Observe that, under our notation, $(\pi_X r)[i]$ is the projection of the tuple $r[i]$ onto X. As a shorthand, we write $r[i][X]$ instead of $(\pi_X r)[i]$.

Functional dependencies. A *functional dependency*, or *FD* for short, is an expression of the form $X \to Y$, where X and Y are finite sets of attributes. We say that $X \to Y$ is *over* a relation schema R if $X \cup Y \subseteq R$. A relation r over a relation schema containing $X \cup Y$ satisfies the FD $X \to Y$ if the values of the attributes in X uniquely determine the values of the attributes in Y. That is, whenever two tuples of r agree on all attributes of X, they must also agree on all attributes of Y. More formally, r satisfies $X \to Y$ if, for all tuple identifiers i and i' in $\mathsf{tids}(r)$, if $r[i][X] = r[i'][X]$ then $r[i][Y] = r[i'][Y]$. For a set Δ of FDs over $\mathbf{Att}(r)$, we denote by $r \models \Delta$ the statement that r satisfies every FD in Δ.

An FD $X \to Y$ is *unary* if X consists of a single attribute, and it is *trivial* if $Y \subseteq X$, in which case it is satisfied by every relation. A *matching constraint* is a constraint of the form $X \leftrightarrow Y$, which is shorthand for the pair $\{X \to Y, Y \to X\}$. An attribute A is a *common left-hand side* (common lhs) of Δ if A occurs in the left-hand side of every FD in Δ.

In our examples, we adopt the standard convention of omitting curly braces and commas when writing sets of attributes. For example, we write AB instead of $\{A, B\}$. We also use a compact notation for sets of FDs by combining multiple dependency expressions and possibly varying the direction of the arrows. For example, $A \leftrightarrow B \leftarrow CD$ is shorthand for the set $\{A \to B, B \to A, CD \to B\}$. The closure of a set Δ of FDs, denoted Δ^+, is the set of all FDs implied by Δ (equivalently, all FDs that can be derived from Δ using Armstrong's axioms). In particular, Δ^+ contains all trivial FDs.

The closure of a finite set X of attributes with respect to Δ, denoted X^+_Δ, is the set of all attributes A such that $X \to A$ belongs to Δ^+. Two finite sets X and Y of attributes are *equivalent* (with respect to Δ) if they have the same closure, that is, if $X^+_\Delta = Y^+_\Delta$ (or, equivalently, $X \to Y$ and $Y \to X$ both belong to Δ^+). By a slight abuse of notation, we say that two attributes A and B are *equivalent* if the singleton sets $\{A\}$ and $\{B\}$ are equivalent.

Finally, for a set Δ of FDs, we denote by $\mathbf{Att}(\Delta)$ the set of all attributes that appear in either the left-hand or right-hand side of an FD in Δ.

Strongly Connected Components (SCCs) of attributes. When Δ is a set of unary FDs, it is convenient to view it as a directed graph over its attributes. Since a relation satisfies Δ if and only if it satisfies its closure Δ^+, the two induce exactly the same repair problem; we therefore define the graph over Δ^+, which is more convenient to work with.

▶ **Definition 2** (FD graph). *Let Δ be a set of unary FDs over R. The* FD graph *of Δ, denoted $\mathcal{G}(\Delta) = (V, E)$, is the directed graph whose node set V is the set $\mathbf{Att}(\Delta)$ of attributes occurring in Δ, and in which $(A, B) \in E$ if and only if $A \neq B$ and $A \to B \in \Delta^+$.*

The *strongly connected components* (SCCs) of Δ are the sets of FDs induced by the SCCs of $\mathcal{G}(\Delta)$. Hence, the SCCs are the equivalence classes of $\mathbf{Att}(\Delta)$ under the aforementioned equivalence relation on attributes: two attributes lie in the same SCC if and only if they are

equivalent, that is, each determines the other. Since $\mathcal{G}(\Delta)$ is transitively closed, the SCCs are the maximal complete directed subgraphs.

The *weakly connected components* of Δ are the sets of FDs induced by the weakly connected components of $\mathcal{G}(\Delta)$; that is, two FDs lie in the same weakly connected component whenever their attributes are joined by a path in the undirected graph underlying $\mathcal{G}(\Delta)$. We say that Δ is *weakly connected* if $\mathcal{G}(\Delta)$ consists of a single weakly connected component.

3 Optimal Update Repairs

In this section, we formally introduce the computational problem studied in this paper, that of computing an *optimal update repair* of an inconsistent relation. We also give an overview of the state of affairs for this problem.

3.1 Problem Definition

As is standard in the study of repairs, we measure complexity in the sense of *data complexity*: the relation schema R and the set Δ of FDs are *fixed*, and the input consists of a single relation r over R.

Updates and optimal U-repairs. Let r be a relation over a schema R. An *update* of r is a relation r' over R obtained from r by modifying attribute values, without inserting or deleting tuples; formally, r' is an update of r if $\mathsf{tids}(r') = \mathsf{tids}(r)$. Note that the values used in an update are not restricted to the *active domain* of r (the values already occurring in r); any value in $\mathbf{Val}$ may be used. Following prior work [13, 16], we define the *distance* from an update r' to r, denoted $dist(r', r)$, as the total number of cells in which r' differs from r:

$$dist(r', r) \stackrel{\text{def}}{=} \sum_{i \in \mathsf{tids}(r)} \left|\{A \in \mathbf{Att}(r) \mid r'[i][A] \neq r[i][A]\}\right|.$$

▶ Remark 3. We may also wish to capture scenarios in which modifying certain cells is more costly than modifying others. To this end, the definition of distance naturally extends to a *weighted* distance, in which each cell is assigned a specific update cost. For clarity of presentation, we focus on the unweighted distance defined above. However, our results immediately extend to this weighted case, as we explain in Section **??**. ◁

Fix a set Δ of FDs over R. A *consistent update* of r (with respect to Δ) is an update r' of r such that $r' \models \Delta$.

▶ **Definition 4** (Optimal update repair). *An* optimal update repair *of r (with respect to Δ), or* optimal U-repair *for short, is a consistent update r' of r that minimizes $dist(r', r)$ among all consistent updates of r; that is, $dist(r', r) \leq dist(r'', r)$ for every update r'' of r with $r'' \models \Delta$.*

The computational problem we study is that of producing such an optimum.

Problem:	*Computing an optimal U-repair for* (R, Δ)
Input:	A relation r over R.
Goal:	Compute an optimal U-repair of r with respect to Δ.

Note that every pair (R, Δ) defines a separate computational problem. A straightforward observation is that R does not play any role in the complexity of this problem, as long as it

contains all of the attributes in $\mathbf{Att}(\Delta)$. Our main result, stated formally as Theorem 5 in Section 3.3, is a complete dichotomy in the complexity of this problem for the case in which Δ consists of unary FDs. Specifically, we classify all such Δ into ones where an optimal U-repair can be found in polynomial time, and ones that induce an NP-hard optimization problem. To present this result, we first need to review the state of affairs for this problem.

3.2 State of Affairs

The complexity of computing optimal repairs by value updates was first studied by Kolahi and Lakshmanan [13] and later, in a systematic way, by Livshits et al. [16]. The latter work develops a rich toolbox for the problem, yet the exact tractability frontier for update repairs has remained elusive; indeed, in contrast to the full dichotomy they establish for *subset* repairs, the authors explicitly leave the complexity of optimal *update* repairs largely open. We summarize below what is known, focusing on unary FDs.

Decomposition. Livshits et al. [16] establish a *decomposition* property: the optimal U-repair problem for Δ can be solved by partitioning Δ into its weakly connected components, computing an optimal U-repair for each component independently, and combining the resulting cell updates, which are guaranteed to touch pairwise disjoint sets of attributes. Consequently, the problem for Δ is solvable in polynomial time whenever it is solvable in polynomial time for every weakly connected component of Δ; conversely, if the problem is NP-hard for even a single weakly connected component, then it is NP-hard for Δ as a whole.

Two tractable cases. Livshits et al. [16] further establish the tractability of two families of FD sets. The first consists of the *chain* FD sets: Δ is a *chain* if its FDs can be arranged so that the left-hand side of each is contained in the left-hand side of the next. For *unary* FDs this criterion is restrictive: since every left-hand side is a single attribute, all FDs of a chain must share the *same* left-hand-side attribute, so the case collapses to Δ having a *common lhs*. The second family is the matching $\{A \to B, B \to A\}$. Combined with the aforementioned decomposition property, this yields the following state of knowledge on the positive side: the optimal U-repair problem is in P if every weakly connected component of Δ has a common lhs or is a matching.

Known intractable case. On the negative side, the only FD set previously known to induce an NP-hard optimal U-repair problem is the directed path $\{A \to B, B \to C\}$, whose hardness was established by Kolahi and Lakshmanan [16]. This single intractable component, together with the decomposition property, is the extent of the known hardness landscape.

Two attempts to enlarge the map of intractable cases have appeared, both of which turned out to be flawed. First, Livshits et al. [15] provided an NP-hardness argument for the FD set $\{A \to B, B \to A, B \to C\}$, but subsequently acknowledged an error in that proof [16]. Second, Miao et al. [18] claimed NP-hardness for the FD set $\{A \to C, B \to C\}$; as we discuss in Section 5.1, their proof contains a flaw.

3.3 Main Result

We make a significant step toward resolving the long-standing open problem of the complexity of U-repairs: we provide a *complete* complexity classification of the optimal U-repair problem for the case of unary FDs. As it turns out, the two polynomial-time cases identified by Livshits et al. [16] are, in fact, *the only* tractable cases (assuming P $\neq$ NP): for unary FDs,

every FD set that is not captured by these two cases gives rise to an NP-hard optimal U-repair problem. Note that trivial FDs never take part in a violation and can be removed from Δ without changing the set of consistent updates of any relation; therefore, we assume throughout that Δ contains no trivial FDs.

▶ **Theorem 5.** *Let Δ be a set of nontrivial unary FDs. An optimal U-repair can be computed in polynomial time if every weakly connected component Δ' of Δ has a common lhs ($A \to X$) or is a matching constraint ($A \leftrightarrow B$). Otherwise, computing an optimal U-repair is NP-hard.*

As aforesaid, the tractable side of Theorem 5, the case in which every weakly connected component has a common lhs or is a matching, has already been settled by Livshits et al. [16]. Our contribution is therefore the hardness side, which we establish for the natural decision variant of the problem: given a relation r and a budget $K \in \mathbb{N}$, decide whether there is a consistent update r' of r with $dist(r', r) \leq K$.

3.4 Proof Strategy

We prove the hardness side of Theorem 5 using two main steps. In the first step, we establish the NP-hardness for several specific cases. One case is where the FD set is a single SCC with three or more attributes (Section 4). Other cases are specific FD sets involving three or four attributes (Section 5). Among the FD sets that we handle are those whose complexity has been left open following published proofs that were found flawed, as we discussed in Section 3.2; hence, this part of the proof resolves these open problems. The second part (Section 6) extends the specific cases of the first part to the general case of a set of unary FDs. We first show that whenever the FD set does not belong to the positive side of Theorem 5, there is a set of attributes, called *convex*, that induces one of the special cases proved hard in the first part. Finally, we show a reduction from the U-repair problem for any convex set of attributes to the original problem (involving all attributes).

4 Three or More Strongly Connected Attributes

We begin our analysis with the case of a strongly connected set of FDs (consisting of a single SCC). Recall that Δ is *strongly connected* if it is a set of pairwise-equivalent attributes. We denote by Δ_c^{sc} the strongly connected FD set with c attributes. (Note that all such FD sets are equivalent.) Specifically, we assume that

$$\Delta_c^{\mathsf{sc}} := \{A_i \to A_j \mid i, j \in \{1, \ldots, c\} \wedge i \neq j\} \quad \text{with} \quad \mathbf{Att}(\Delta_c^{\mathsf{sc}}) = \{A_1, \ldots, A_c\}\,.$$

▶ **Theorem 6.** *For every $c \geq 3$, computing an optimal U-repair for Δ_c^{sc} is NP-hard.*

The proof of Theorem 6 is nontrivial, and the rest of this section sketches the proof. The complete construction and formal analysis are provided in the Appendix. Importantly, the combinatorial core of the proof lies in the case of three attributes. We establish this case by a reduction from a restricted variant of three-dimensional matching, and we then reduce it to Δ_c^{sc} for every $c > 3$ by padding each tuple with values that no repair within the budget can preserve. Hence, the challenge of the proof is concentrated on the case of $c = 3$, and it is the case that we sketch here.

The fact that all attributes in Δ_c^{sc} determine one another imposes a strict structure on consistent relations. If two tuples of a relation $r \models \Delta_c^{\mathsf{sc}}$ agree on even a single attribute A_i, then they must agree on all attributes $A_1, \ldots, A_c$. Equivalently, every two tuples are either completely identical on $A_1, \ldots, A_c$, or they disagree on every single one of them.

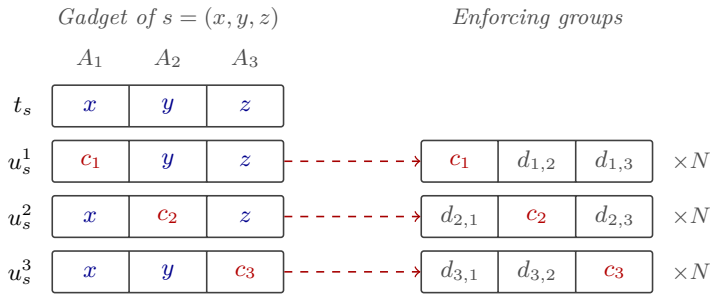

Figure 1 The tuples that the reduction introduces for a triple $s = (x, y, z)$. Each auxiliary tuple inherits two values from the S-tuple and contains a fresh constant in the remaining attribute; the dashed arrows lead to the enforcing group that anchors this constant.

Consequently, a consistent relation partitions its tuples into clusters of identical tuples. Each cluster is defined by a single *target tuple* $v \in \mathbf{Val}^c$ shared by all its members, and the assignments of any two distinct clusters must disagree in all c coordinates. Repairing r is thus the act of assigning a target tuple v to every tuple: a tuple t modified to match v is charged one unit for each coordinate on which t and v differ (that is, c minus the number of original cells it retains). Computing an optimal U-repair for Δ_c^{sc} is therefore the problem of partitioning the tuples into these disjoint clusters and modifying each cluster to a joint v, which is unique on every attribute, while preserving as many original cells as possible.

Restricted to three attributes, this clustering problem is closely related to the *3-Dimensional Matching* (3DM) problem. Consider the values that occur in these three attributes as elements of three disjoint sets X, Y, and Z, so that every tuple is a triple in $X \times Y \times Z$. In 3DM, one is given such sets, each of size q, and a set $S \subseteq X \times Y \times Z$ of triples, and the goal is to select a subset of S that touches every element of $X \cup Y \cup Z$ exactly once; equivalently, we ask whether we can find q pairwise-disjoint triples. Our repair problem has a similar shape: a repair, too, selects a set of target triples that must be pairwise disjoint, and it pays nothing for a tuple it leaves unchanged and one unit for each cell it modifies. However, in 3DM, the chosen triples must belong to S, whereas a repair may use *any* triple of $\mathbf{Val}^3$ as a target triple. To this aim, we define a restricted form of 3DM.

Restricted matching problem (R3DM). The restricted variant we reduce from, R3DM, imposes two structural conditions on the input triples of 3DM:

- Limited Intersection (LI): every two distinct triples share at most one element;
- Triangle-freeness (TF): there are no three distinct triples and three elements x, y, z such that one triple contains $\{x, y\}$, another $\{x, z\}$, and the third $\{y, z\}$.

We prove in the Appendix that R3DM is still NP-complete. As we explain later, our reduction relies heavily on the LI and TF properties for correctness, specifically to show that an optimal U-repair yields a perfect matching.

Reduction gadget. Next, we describe our reduction, which is illustrated in Figure 1. Fix an R3DM instance. For each triple $s = (x, y, z)$ in the instance, we build a gadget on the three attributes, as shown in Figure 1.

- An *S-tuple* (x, y, z), that is, s itself;
- Three *auxiliary tuples*: the auxiliary tuple for a given attribute agrees with the S-tuple on the other two attributes, but contains a fresh constant in its own attribute.

We refer to these four tuples as the *gadget tuples*. Finally, to prevent fresh constants from being chosen as part of a target triple, each fresh constant is anchored by an *enforcing group*: a set of N identical tuples that agree with the auxiliary tuple on the fresh constant and carry, in every other attribute, a value that occurs nowhere else. We set the budget to $K := 8m - 5q$ and the size of the enforcing groups to $N := K + 1$. All in all, the constructed relation r consists of the $4m$ tuples of the m gadgets and the $3m$ enforcing groups.

Correctness We first show that a perfect matching yields a U-repair. This direction is fairly simple. Given a perfect matching $M \subseteq S$, we use the triples of M as the target tuples, and update every gadget tuple to be equal to a target tuple that is closest to it. (In general, finding a low-cost U-repair for Δ_c^{sc} boils down to finding a good set of target tuples.)

Next, we show that an in-budget U-repair yields a perfect matching. This direction is considerably more challenging. Consider a consistent update r' of cost at most K. Every two target tuples are disjoint. We show that r' has at least q clusters where each has a target tuple that is an S-tuple (i.e., a triple of the R3DM instance). Hence, these q target tuples form a perfect matching.

To this end, we first show that the enforcing groups prevent the gadget tuples from retaining their fresh constants. Moreover, r' has no reason to change the enforcing tuples. Therefore, the cost of r' is the number of cells changed within the gadget tuples. We also show that there is no reason to update more than two cells in each tuple of r. Hence, the cost c_t incurred by each individual gadget tuple t is at most two. For convenience, we refer to $2 - c_t$ as the *saving* of t. We further refer to the *saving of a cluster* as the total savings of all tuples in the cluster. With zero savings, we have $4m$ tuples with a total cost of $8m$. Since the total cost of r' is at most $K = 8m - 5q$, we conclude that the total savings over all clusters are at least $5q$.

From here on, we analyze the different clusters of r' according to properties of their target tuples v. There are four types:

(a) No pair of values in v co-occurs in any triple of S.

(b) Exactly one pair in v co-occurs in a triple of S.

(c) Exactly two pairs in v co-occur in triples of S.

(d) v is an S-tuple.

Due to the TF property, these types are pairwise disjoint, and they cover all possibilities of clusters of r'. We show a bound on the savings of each cluster, according to its type. A numerical analysis then shows that, to reach the savings of $5q$ (or more), there can be no clusters of type (a)-(c), and there are precisely q clusters of type (d). As aforesaid, the corresponding q target tuples form a perfect matching, as required.

Beyond three attributes. To extend this construction to relations of arity c, we select three arbitrary attributes to carry the R3DM gadgets and their matching logic. For each of the remaining $c - 3$ attributes, we assign a unique fresh constant to every tuple, and guard this constant with a dedicated enforcing group. This forces every consistent update within the budget to modify all values in these additional attributes, at a fixed, uniform cost. Since this extra cost is independent of the other choices, it merely shifts the budget by a constant, leaving the equivalence with the existence of a perfect matching unaffected.

5 Case Analysis for Small SCCs

Theorem 6 settles the FD sets that are strongly connected and have three or more attributes. In this section, we turn to the FD sets in which every SCC of $\mathcal{G}(\Delta)$ consists of one or two attributes. An SCC of a single attribute induces no nontrivial FDs at all, and an SCC of two attributes A and B induces the matching constraint $A \leftrightarrow B$, which is one of the two tractable cases of Theorem 5. Hence, unlike the situation of Section 4, intractability can no longer originate in a single SCC; it can arise only from the way in which several SCCs are *combined*, that is, from the FDs that hold between attributes of distinct SCCs. The FD sets that neither section covers, namely those that have an SCC of three or more attributes without being strongly connected themselves, are handled in Section 6, where we show that an FD set inherits the hardness of a convex subset and, in particular, that of any of its SCCs.

We establish hardness for the following combinations of small SCCs:

- $A \rightarrow B \rightarrow C$: three single-attribute SCCs forming a directed path;
- $A \rightarrow B \leftarrow C$: two single-attribute SCCs that determine a third;
- $A \leftrightarrow B \rightarrow C$: a matching constraint that determines a single attribute;
- $A \rightarrow B \leftrightarrow C$: a single attribute that determines a matching constraint;
- $A \leftrightarrow B \rightarrow C \leftrightarrow D$: a matching constraint that determines another matching constraint.

The first combination, the directed path, is already known to be intractable [13], as discussed in Section 3.2, so we prove hardness for the remaining four. We give the full proof for $A \rightarrow B \leftarrow C$; for the other three, we give only a proof sketch here, and the complete construction and formal analysis are provided in the appendix.

5.1 Hardness of $A \rightarrow B \leftarrow C$

We begin with $A \rightarrow B \leftarrow C$, that is, the FD set $\{A \rightarrow B, C \rightarrow B\}$, in which two independent attributes determine a third. As mentioned in Section 3.2, Miao et al. [18] have already claimed that this FD set is intractable. They give two arguments, and we begin by explaining why each of them is flawed.

The first argument is based on the *conflict hypergraph* of a relation r: the vertices are the cells of r, and the hyperedges are the sets of cells that jointly witness a violation of an FD of Δ. The authors construct a reduction *from* the optimal U-repair problem *to* minimum vertex cover over conflict hypergraphs, and intractability is then deduced from the NP-hardness of the latter. This deduction reverses the direction of the reduction. Observe also that the hardness argument is stated for every Δ that contains two FDs $X \rightarrow C$ and $Y \rightarrow C$ with $X \neq Y$, and in this generality it cannot hold (unless P = NP): the FD set $\{A \rightarrow C, AB \rightarrow C\}$ satisfies this condition, yet its left-hand sides form a chain, since $A \subseteq AB$, and so an optimal U-repair for it can be computed in polynomial time [16].

The second argument is a direct reduction from Max NM-E3SAT, the problem of satisfying as many clauses as possible of a 3CNF formula φ in which every clause consists of either three positive literals or three negative literals. Given such a formula with m clauses, the constructed relation over $\{A, B, C\}$ contains, for every clause c and every variable x occurring in c, the tuple $(c, 1, x)$ if x occurs positively in c, and the tuple $(c, 0, x)$ if it occurs negatively. The analysis concludes that the minimum cost of a consistent update of this relation is precisely $3m - s$, where s is the maximum number of clauses of φ that a single assignment satisfies; as $s \leq m$, this cost is at least $2m$. The conclusion is false. Setting the B-value of *every* tuple to 1 makes the B-column constant and, therefore, already produces a consistent relation; its cost is the number of tuples that carry 0, that is, three times the

number of all-negative clauses. Symmetrically, setting every B-value to 0 costs three times the number of all-positive clauses. One of the two is of cost at most $3m/2$, which is strictly below the claimed minimum.

We now establish the hardness of $A \to B \leftarrow C$.

▶ **Proposition 7.** *Computing an optimal U-repair for $A \to B \leftarrow C$ is NP-hard.*

Proof. We reduce from the multiway-cut problem with three terminals on bipartite graphs. Let $G = (U, W, E)$ be a bipartite graph with sides U and W, and let $S = \{s_1, s_2, s_3\} \subseteq U \cup W$ be three distinct *terminals*. A *multiway cut* of (G, S) is a set $E' \subseteq E$ such that no two terminals are connected in the graph obtained from G by deleting the edges of E'. Deciding whether (G, S) has a multiway cut of size at most k is NP-complete already for three terminals [8]. It remains NP-complete on bipartite graphs: subdividing every edge once (i.e., replacing each edge $u - v$ by a path $u - x_{uv} - v$ through a new vertex x_{uv}) makes the graph bipartite, changes neither the terminals nor the minimum size of a multiway cut, and preserves NP-completeness [11].

Construction. Let (G, S, k) be such an instance, with G bipartite, and denote $m = |E|$ and $n = |U| + |W|$. We may assume that $k \leq m$, since otherwise E itself is a multiway cut of size at most k. We construct a relation r, as illustrated in Figure 2, over $\{A, B, C\}$ and a budget K_r, writing a tuple as a triple (a, b, c) that lists its values for A, B and C. The attribute B holds one of three *colors* 1, 2 and 3, one for each terminal. We set $H := 2k + 1$. The relation r consists of the following tuples.

- *Anchors.* If $v \in U \cup W$ and $b \in \{1, 2, 3\}$, then we call the tuple (v, b, v) an *anchor* of v. We add to r:
  - H anchors for every vertex v and color $i \in \{1, 2, 3\}$ (hence, $3H$ tuples in total for v).
  - Additional H anchors (s_i, i, s_i) for every terminal $s_i \in \{s_1, s_2, s_3\}$.
- *Edge gadgets.* For every edge $\{u, w\} \in E$ with $u \in U$ and $w \in W$, the relation contains the three tuples $(u, 1, w)$, $(u, 2, w)$ and $(u, 3, w)$.

The FD set Δ requires that, whenever two tuples agree on A or on C, they should also agree on the color in B. Hence, the multiplicity of colors is what causes the violations in r.

Finally, we set $K_r := 2nH + 2m + 2k$. Both r and K_r are clearly computable in polynomial time. We need to show that (G, S) has a multiway cut of size at most k if and only if r has a consistent update r' with $dist(r', r) \leq K_r$.

From a multiway cut to a consistent update. Let E' be a multiway cut of (G, S) with $|E'| \leq k$. Every connected component of the graph $G' = (U \cup W, E \setminus E')$ contains at most one terminal, so we obtain a coloring $\lambda : U \cup W \to \{1, 2, 3\}$ by setting $\lambda(v) = i$ if the component of v contains s_i, and $\lambda(v) = 1$ if it contains no terminal. In particular, $\lambda(s_i) = i$, and $\lambda(u) = \lambda(w)$ for every remaining edge $\{u, w\} \in E \setminus E'$.

We construct r' as follows. For every vertex v, we change to $\lambda(v)$ the color of every anchor of v whose color differs from $\lambda(v)$. A vertex $v \notin S$ has $2H$ such anchors that we update, and so does a terminal s_i, since $\lambda(s_i) = i$ and there are $2H$ anchors of s_i of the other two colors. The anchors therefore cost $2nH$ in total. Next, consider the gadget of an edge $\{u, w\}$.

- If $\lambda(u) = \lambda(w)$, then we change to $\lambda(u)$ the color of the two tuples of the gadget whose color differs from $\lambda(u)$; the gadget becomes three copies of $(u, \lambda(u), w)$, at a cost of 2.
- If $\lambda(u) \neq \lambda(w)$, then the edge $\{u, v\}$ is necessarily one of the removed tuples in E'. We replace the C-value of the tuple of color $\lambda(u)$ with a fresh value, we replace the A-value of the tuple of color $\lambda(w)$ with a fresh value, and we replace both the A-value and the

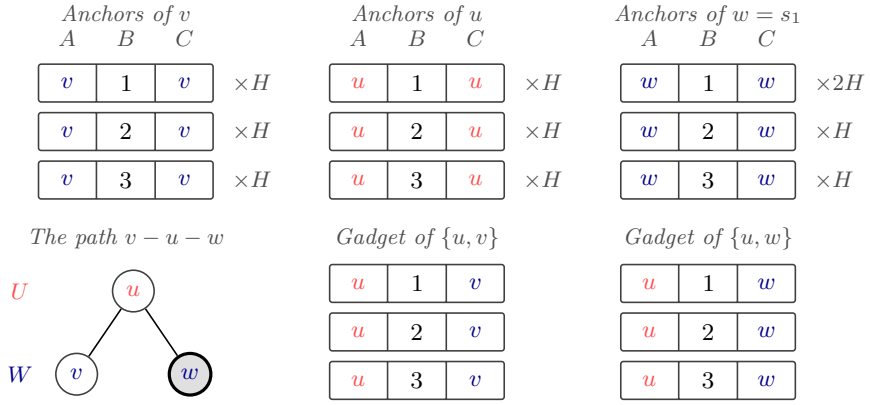

Figure 2 The tuples that the reduction constructs for a path $v - u - w$ of G, where $u \in U$, $v, w \in W$, and w is the terminal s_1.

C-value of the third tuple with fresh values, where every fresh value is used in a single cell of r'. The cost is $1 + 1 + 2 = 4$.

Thus, every edge incurs a cost of 2, and each edge in E' may incur 2 more. Since $|E'| \leq k$, we conclude that the total cost for updates in the edge gadgets is at most $2m + 2k$. We conclude that $dist(r', r) \leq 2nH + 2m + 2k = K_r$, as required.

It remains to show that $r' \models \{A \to B, C \to B\}$. Let a be a value that occurs in the A-column of r'. If $a = v$ for a vertex $v \in U \cup W$, then the tuples with $A = v$ are the anchors of v together with the tuples of gadgets of edges incident to v that retained their A-value, and all of them have the color $\lambda(v)$. Every other value of the A-column is fresh and occurs in a single tuple. Hence, $r' \models A \to B$. The argument for $C \to B$ is symmetric.

From a consistent update to a multiway cut. Let r' be a consistent update of r with $dist(r', r) \leq K_r$. We say that a tuple of r' is *attached* to a vertex $u \in U$ if its A-value is u, and that it is attached to a vertex $w \in W$ if its C-value is w. Since $r' \models A \to B$ and $r' \models C \to B$, all the tuples that are attached to a vertex v agree on their B-value; we denote this value by $\lambda(v)$. If no tuple is attached to a node v, then we let $\lambda(v)$ be a value that is not a color, say $\diamond$.

We first bound the cost of the anchors. If an anchor of v is unchanged in r', then it is attached to v, and its color is $\lambda(v)$. Consequently, the anchors of v contribute to $dist(r', r)$ at least the number of anchors of v whose color differs from $\lambda(v)$. This number is:

- $2H$ if $v \notin S$ and $\lambda(v)$ is a color, or $v \in S$ is the terminal s_i and $\lambda(v) = i$.
- $3H$ or more in every other case, that is, $\lambda(v) = \diamond$ or $v = s_i$ and $\lambda(v) \neq i$.

Summing over the n vertices, the anchors cost at least $2nH$, and at least $2nH + H$ if some vertex v has a $\lambda(v)$ that is not a color or some terminal s_i has $\lambda(s_i) \neq i$.

We now bound the cost of the edge gadgets. The three tuples of the gadget of $\{u, w\}$ agree on A and on C but have distinct colors, so at most one of them is unchanged in r', and the gadget costs at least 2. Therefore, the edge gadgets contribute at least $2m$ to the cost.

Hence, the anchors and the gadgets cost at least $2nH + 2m$ altogether, which leaves a slack of $K_r - (2nH + 2m) = 2k$. Since $H = 2k + 1$, the additional cost of H identified above exceeds this slack, and it follows that λ maps every vertex to a color and that $\lambda(s_i) = i$ for $i = 1, 2, 3$. In particular, every vertex has at least one attached tuple in r', since otherwise its λ-value would not be a color.

Let E' be the set of the edges $\{u, w\} \in E$ with $\lambda(u) \neq \lambda(w)$. We claim that the gadget of such an edge costs at least 4. No tuple of r' has both $A = u$ and $C = w$, since its color would then be both $\lambda(u)$ and $\lambda(w)$; therefore, each of the three tuples of the gadget changes its A-value or its C-value. Moreover, the tuple t whose color is neither $\lambda(u)$ nor $\lambda(w)$ changes at least two cells: if t retains $A = u$, then it changes its color to $\lambda(u)$ and, in addition, its C-value; if t retains $C = w$, then it changes its color to $\lambda(w)$ and, in addition, its A-value; and if t retains neither, then it changes both its A-value and its C-value.

We conclude that $dist(r', r) \geq 2nH+2m+2|E'|$, and $|E'| \leq k$ follows from $dist(r', r) \leq K_r$. Finally, E' is a multiway cut of (G, S): the endpoints of every edge of $E \setminus E'$ have the same color, hence every connected component of $(U \cup W, E \setminus E')$ is monochromatic, whereas the terminals have pairwise distinct colors (i.e., $\lambda(s_i) = i$). ◀

5.2 Remaining Proofs

We now turn to the three remaining combinations, describing for each of them the construction and the idea behind its analysis.

▶ **Proposition 8.** *Computing an optimal U-repair for $A \leftrightarrow B \rightarrow C$ is NP-hard.*

Proof. *(Sketch)* We reduce from Vertex Cover. Let $G = (V, E)$ be a graph and let $k \leq |V|$ be a bound on the size of a vertex cover. We set $L := |V| + 1$ and $M := L|E| + |V| + 1$, and we construct a relation r over $\{A, B, C\}$ that consists of:

- The *anchors* of every vertex $v \in V$, consisting of $M + 1$ copies of the tuple $(v, v, 0)$ and M copies of $(v, v, 1)$;
- An *edge gadget* of L copies of $(u, v, 1)$ for every edge $\{u, v\} \in E$. (We select an arbitrary ordering (u, v) between the nodes.)

The budget is $K_r := M|V| + L|E| + k$.

Given a vertex cover S of size at most k, we construct a repair by changing every anchor $(v, v, 0)$ to $(v, v, 1)$ if $v \in S$ (cost $M|S| + |S|$), changing every anchor $(v, v, 1)$ to $(v, v, 0)$ if $v \notin S$ (cost $M(|V| - |S|)$), and changing every edge gadget $(u, v, 1)$ to $(u, u, 1)$ or $(v, v, 1)$, depending on whether u or v is in S (cost $L|E|$). All in all, the cost is $M|V|+L|E|+|S| \leq K_r$.

Now suppose that we are given a repair of cost at most K_r. Since $A \leftrightarrow B$, the pairs of A-values and B-values that occur in a consistent relation form a partial bijection. The anchors of v all carry the pair (v, v) and split into a group of $M + 1$ tuples with $C = 0$ and a group of M tuples with $C = 1$, violating $B \rightarrow C$. Hence, we must modify at least one cell in every tuple of one of them, at a cost of at least M per vertex. The budget is $M|V|+L|E|+k$, so this leaves a slack of $L|E| + k$. Pulling the two groups apart is expensive: a group can be separated only by changing *both* its A-value and its B-value, since changing one of the two alone collides with the group that stays, and this costs at least $2M$. As M exceeds the entire slack, no repair can afford it, and every vertex must instead unify its anchors on a single value $c_v \in \{0, 1\}$ of C, at a cost of M if $c_v = 0$ and of $M + 1$ if $c_v = 1$. The anchors therefore cost precisely $|V|M + |S|$, where $S := \{v \in V \mid c_v = 1\}$ is the intended vertex cover.

The edge gadgets test whether S covers E. A tuple $(u, v, 1)$ cannot retain both $A = u$ and $B = v$, since the anchors bind u to the pair (u, u) and v to the pair (v, v); it must join the cluster of u or the cluster of v. Joining that of u amounts to changing its B-value, and it changes nothing else exactly when $c_u = 1$, and symmetrically for v. Hence, an edge costs L if at least one of its endpoints belongs to S, and at least $2L$ otherwise. Since $L > k$, the budget admits no uncovered edge, and it leaves $|S| \leq k$. ◀

▶ **Proposition 9.** *Computing an optimal U-repair for $A \to B \leftrightarrow C$ is NP-hard.*

Proof. *(Sketch)* We reduce from 3SAT, assuming that every clause consists of three literals over three distinct variables. Let φ be a formula with variable set V and m clauses, and set $L := 4m + 1$. With every variable v we associate two fresh values T_v and F_v, one for each truth value. The constructed relation over $\{A, B, C\}$ consists of a *variable gadget* for every variable v, namely L copies of (v, v, T_v) and L copies of (v, v, F_v), and a *clause gadget* for every clause $c = (\ell_1 \vee \ell_2 \vee \ell_3)$ over the variables v_1, v_2, v_3, namely the three tuples:

$$(c, v_1, w_3) \qquad (c, v_2, w_1) \qquad (c, v_3, w_2)$$

Here, w_i is the value of v_i that satisfies ℓ_i, that is, T_{v_i} if ℓ_i is positive and F_{v_i} if ℓ_i is negative. Note that the indices in a clause gadget are shifted: v_1, v_2 and v_3 coincide with w_3, w_1 and w_2, respectively. For example, if c is $x \vee y \vee \neg z$, then the three tuples are:

$$(c, x, F_z) \qquad (c, y, T_x) \qquad (c, z, T_y)$$

The budget is $K := L|V| + 4m$. The proof shows how a satisfying assignment can yield a repair that costs L on each variable gadget (changing the truth values according to the assignment) and 4 for each clause gadget. For the other direction, we show that with a budget of K, the variable gadgets must define an assignment τ as in the first direction, and each clause must spend precisely 4 updates, implying that it retains at least one of its satisfying w_i's, which is delivered by the assignment τ. ◀

▶ **Proposition 10.** *Computing an optimal U-repair for $A \leftrightarrow B \to C \leftrightarrow D$ is NP-hard.*

Proof. *(Sketch)* We reduce from $A \leftrightarrow B \to C$, which is the FD set of Proposition 8 and consists of the first three attributes of $A \leftrightarrow B \to C \leftrightarrow D$. Let r be a relation over $\{A, B, C\}$ with n tuples, let K be a budget, and set $N := K + n + 1$. We construct a relation r' over $\{A, B, C, D\}$ that consists of:

- A *main tuple* for every tuple of r, which copies its A-, B-, and C-values and uses a fresh value in the new attribute D, called the *private* value of the tuple;
- An *enforcing group* of N identical tuples for every fresh value d, using the value d in D and, in each of A, B and C, a value that occurs nowhere else.

The budget is $K + n$.

Given a consistent update s of r of cost at most K, we construct a consistent update of r' of cost at most $K + n$: we leave the enforcing groups unchanged, we modify the A-, B- and C-cells of the main tuples as s does, at a cost of at most K, and we give the main tuples that share a C-value in s a common fresh D-value, which satisfies $C \leftrightarrow D$ at the price of one cell per tuple of r. Since these D-values are fresh, no main tuple shares a value with an enforcing group in any attribute.

Now suppose that we are given a consistent update of r' of cost at most $K + n$. As N exceeds the budget, every enforcing group retains an unchanged copy. We may assume that no main tuple agrees with an enforcing tuple on C (equivalently, on D). Indeed, consider a group of main tuples that have been updated to the same (a, b, c, d), where (c, d) is the pair of an enforcing tuple. All of them have changed their C-value, since c occurs nowhere in r, and at most one of them retains its original D-value. We take the original C-value c' of one of them, and we pair it with the D-value already used for c' in the update, if c' already occurs, or with a fresh D-value otherwise. We replace all of their pairs (c, d) by this pair (c', d'). If they also carry the A- and B-values of the enforcing tuple, then they have all been

updated, and we replace them by a fresh pair; this does not increase the cost since these cells were already changed. One C-cell is then restored and at most one extra D-cell is changed, so the cost does not increase. Repeating for every such group, we get a repair of r' where no main tuple agrees with an enforcing tuple on D.

It follows that no main tuple retains its private value, since it would then agree with its own enforcing tuple on D. The update therefore modifies all the D-cells of the main tuples and at most K cells on A, B and C, and their restriction to A, B and C is an update of r of cost at most K, which is consistent since $A \leftrightarrow B \rightarrow C$ is implied by $A \leftrightarrow B \rightarrow C \leftrightarrow D$. ◀

6 Generalization via Hard Convex Sets

We now generalize the hardness results of the specific cases from the previous sections to the full hardness side of Theorem 5, thus completing the proof of the theorem. For the rest of this section, we fix a set Δ of unary FDs.

For a set S of attributes, we denote by Δ_S the FD set that S *induces*, that is, the set of all FDs in Δ^+ that use only attributes in S. The generalization goes as follows. We first define the notion of a *convex* set S of attributes with respect to Δ. Second, we show that for each convex set S of Δ, there is a reduction from computing an optimal U-repair for Δ_S to computing an optimal U-repair for Δ. Third, we show that if Δ is in the hardness side of Theorem 5, then at least one of its convex sets S induces one of the hard special cases Δ_S of the previous sections. We begin with the definition of convexity.

▶ **Definition 11** (Convexity). *A subset S of $attr(\Delta)$ is said to be* convex *(*w.r.t. Δ*) if for every three attributes A, B and C with $A \rightarrow B \rightarrow C \in \Delta^+$, if $A \in S$ and $C \in S$ then $B \in S$.*

As an example, consider $\Delta = \{A \rightarrow B, B \rightarrow C, A \rightarrow D, D \rightarrow C\}$. Then $S = \{A, B, C\}$ is *not* convex, since we have $A \rightarrow D \rightarrow C \in \Delta^+$ and both A and C are in S, but D is not. On the other hand, the sets $\{A\}$, $\{A, B\}$ and $\{A, B, D\}$ are all convex. Also, convexity is closely related to strong connectivity:

▶ **Observation 12.** *The following hold.*

1. *Every SCC is convex.*
2. *Every convex set is the union of SCCs (but a union of SCCs is not necessarily convex).*

We prove the following:

▶ **Lemma 13** (Reduction from a convex set). *Let Δ be a set of unary FDs, and let $S \subseteq attr(\Delta)$ be a convex set of attributes. There is a polynomial-time cost-preserving reduction from computing an optimal U-repair for Δ_S to computing an optimal U-repair for Δ.*

Proof. We fix a constant $\diamond$ in **Val** that we use throughout the proof. An attribute $B \in R \setminus S$ is said to be *determined* by S if $A \rightarrow B \in \Delta^+$ for some $A \in S$. Given a relation s over S, we construct a relation r over $R = (\Delta)$ with $\mathsf{tids}(s) = \mathsf{tids}(r)$ by setting, for every $i \in \mathsf{tids}(r)$:

- $r[i][A] = s[i][A]$ for each attribute $A \in S$;
- $r[i][B] = \diamond$ for each attribute B determined by S;
- $r[i][C]$ is a fresh new value for each attribute that is neither in S nor determined by S.

The construction is *correct* in the sense that to repair r, it is necessary and sufficient to repair the projection of r to S, namely s. Hence, a U-repair for r yields a U-repair (of the same or smaller cost) for s, and a U-repair for s yields a U-repair (of the same cost) for r.

We prove correctness as follows. Let i and j be identifiers in $\mathsf{tids}(r)$. We need to show that every FD $A \to B$ violated by $\{r[i], r[j]\}$ is such that both A and B are in S (hence, $\Delta_S \models A \to B$). Thus, it suffices to show that if either $A \notin S$ or $B \notin S$, then the FD $A \to B$ is satisfied, regardless of the content of s. (In particular, it continues to be satisfied even if s is updated.) We consider several cases.

1. If $B \in S$, then $A \notin S$ and A cannot be determined by S, or otherwise A would need to be in S since we assume that S is convex. Therefore, the rows disagree on A because $r[i][A]$ and $r[j][A]$ are fresh new values (by construction).
2. If B is determined by S, then the rows agree on B, since $r[i][B] = r[j][B] = \diamond$.
3. If B is neither in S nor determined by S, then A is also neither in S nor determined by S, or otherwise $A \to B$ would imply that B is determined by S. Hence, the rows disagree on A as in the first case.

In each of the three cases, the FD $A \to B$ is satisfied, as claimed. ◀

From Observation 12 and Lemma 13, we immediately conclude that it is NP-hard to compute an optimal U-repair for Δ if it is NP-hard for at least one SCC of Δ. The other direction, however, is false (assuming P $\neq$ NP), as evident from the hardness of $A \to B \to C$, where each SCC consists of a single attribute.

Finally, we show that the hard cases of the previous sections are exhaustive, in the sense that every FD set on the hardness side of Theorem 5 has a convex set of attributes that induces one of them. This, together with Lemma 13, completes the proof of Theorem 5.

▶ **Lemma 14** (Existence of a hard convex set)**.** *Let Δ be a set of nontrivial unary FDs that does not fall in the tractable side of Theorem 5. Then Δ has a convex set S of attributes such that, up to a renaming of the attributes, Δ_S is one of the FD sets $A \to B \to C$, $A \to B \leftarrow C$, $A \leftrightarrow B \to C$, $A \to B \leftrightarrow C$, $A \leftrightarrow B \to C \leftrightarrow D$, and Δ_{K_c} for some $c \geq 3$.*

Proof. Without loss of generality, we assume that Δ is weakly connected. Otherwise, we apply the proof to a weakly connected component of Δ where the tractability condition is violated. Clearly, an attribute set that is convex w.r.t. a weakly connected component of Δ is also convex w.r.t. Δ.

We use the *SCC DAG* of Δ: its nodes are the SCCs of Δ, and it has an edge from a node P to a node Q whenever $P \neq Q$ and the attributes of P determine those of Q. Note that the SCC DAG is connected, since we assume that Δ is weakly connected. We write $P \prec Q$ to state that the SCC DAG has an edge from P to Q. Note that $\prec$ is transitive, asymmetric, and irreflexive, hence a strict partial order. In particular, every nonempty set of SCCs has a $\prec$-minimal member and a $\prec$-maximal member.

We say that a node R is *between* P and Q if $P \prec R \prec Q$. Note that a union of SCCs is convex if and only if no SCC outside the union is between two SCCs in the union. Indeed, if $\Delta \models A \to B \to C$ and A and C belong to the union, then the SCC of B is either that of A or C, or between them (hence, in the union).

Since Δ violates the condition of Theorem 5, it has at least three attributes. We consider three cases, according to the maximal number of attributes in an SCC of Δ.

Case 1: *Some SCC P has three or more attributes.* By Observation 12, the set P is convex, and Δ_P is Δ_{K_c} for $c = |P| \geq 3$.

Case 2: *The maximal number of attributes in an SCC is two.* Let P be an SCC with two attributes, denoted A and B. Then Δ_P is $A \leftrightarrow B$.

Suppose first that $P \prec Q'$ for some SCC Q'. Let Q be a $\prec$-minimal member of the set of nodes Q' with $P \prec Q'$. The set $S := P \cup Q$ is convex: a node between P and Q would belong to the set from which we chose Q and would precede Q, contradicting the minimality of Q.

- If Q consists of a single attribute C, then Δ_S is $A \leftrightarrow B \rightarrow C$.
- If Q consists of two attributes C and D, then Δ_S is $A \leftrightarrow B \rightarrow C \leftrightarrow D$.

If $Q' \prec P$ for some SCC Q', then we select Q as a $\prec$-maximal member of the set of the nodes Q' with $Q' \prec P$. The set $S := P \cup Q$ is again convex.

- If Q consists of a single attribute C, then Δ_S is $C \rightarrow A \leftrightarrow B$.
- If Q consists of two attributes C and D, then Δ_S is $C \leftrightarrow D \rightarrow A \leftrightarrow B$.

Case 3: *Every SCC has a single attribute.* In this case, the SCC DAG is the same as $\mathcal{G}(\Delta)$, identifying each singleton with its single member. Hence, $\mathcal{G}(\Delta)$ is acyclic and weakly connected. Fix a topological order over the attributes.

We first claim that at least one attribute C has two or more incoming edges. Let A' be the first (leftmost) node in the topological order. If A' has an outgoing edge to every other attribute, then at least one neighbor C of A' has an incoming edge from an attribute that is not A', or else Δ would be equivalent to $A' \rightarrow X$ (satisfying the tractability condition). Otherwise, if A' does not have an outgoing edge to every other attribute, then at least one neighbor C of A' has an incoming edge from an attribute that is not A', since $\mathcal{G}(\Delta)$ is connected and transitively closed. This proves the claim.

Choose an arbitrary node C with two or more incoming edges in $\mathcal{G}(\Delta)$. Let B be the rightmost attribute in the topological order with an edge to C. Let A be the rightmost attribute, different from B, with an edge to C. Let $S = \{A, B, C\}$. Hence, the FD set Δ_S is either $A \rightarrow B \rightarrow C$ or $A \rightarrow C \leftarrow B$, depending on whether $A \rightarrow B$ or not. To complete the proof, we need to show that S is convex. This holds due to the choice of B and A as rightmost attributes: if $D \notin S$ is such that if $B \rightarrow D \rightarrow C$, then D should have been chosen instead of B; and if $A \rightarrow D \rightarrow B$ or $A \rightarrow D \rightarrow C$, then D should have been chosen instead of A. ◀

With Lemma 14, we have now completed the proof of Theorem 5.

7 Conclusions

We have established a complete complexity classification for computing optimal U-repairs under unary FDs. Our result shows that the previously known tractable cases are exhaustive: an optimal U-repair can be computed in polynomial time when every weakly connected component of the FD set has a common lhs or is a matching constraint, and the problem is NP-hard in every other case. Thus, for unary FDs, the boundary between tractability and intractability is now fully understood.

The main question left open is whether a similar classification can be obtained for arbitrary FDs. The unary case already exhibits a rich interaction between the structure of the dependencies and the complexity of repairing the data, and several of our hardness arguments rely on phenomena that extend beyond unary FDs. It would be interesting to understand whether these techniques, together with the tractability tools developed in previous work, can lead to a complete dichotomy for optimal U-repairs under general FD sets.

References

1. Foto N. Afrati and Phokion G. Kolaitis. Repair checking in inconsistent databases: algorithms and complexity. In *ICDT*, pages 31–41. ACM, 2009.
2. Marcelo Arenas, Leopoldo E. Bertossi, and Jan Chomicki. Consistent query answers in inconsistent databases. In *PODS*, pages 68–79. ACM, 1999.
3. George Beskales, Ihab F. Ilyas, and Lukasz Golab. Sampling the repairs of functional dependency violations under hard constraints. *Proc. VLDB Endow.*, 3(1):197–207, 2010.
4. Philip Bohannon, Michael Flaster, Wenfei Fan, and Rajeev Rastogi. A cost-based model and effective heuristic for repairing constraints by value modification. In *SIGMOD Conference*, pages 143–154. ACM, 2005.
5. Marco A. Casanova, Ronald Fagin, and Christos H. Papadimitriou. Inclusion dependencies and their interaction with functional dependencies. *J. Comput. Syst. Sci.*, 28(1):29–59, 1984.
6. Jan Chomicki and Jerzy Marcinkowski. Minimal-change integrity maintenance using tuple deletions. *Inf. Comput.*, 197(1-2):90–121, 2005.
7. Gao Cong, Wenfei Fan, Floris Geerts, Xibei Jia, and Shuai Ma. Improving data quality: Consistency and accuracy. In *VLDB*, pages 315–326. ACM, 2007.
8. E. Dahlhaus, D. S. Johnson, C. H. Papadimitriou, P. D. Seymour, and M. Yannakakis. The complexity of multiterminal cuts. *SIAM Journal on Computing*, 23(4):864–894, 1994. doi:10.1137/S0097539792225297.
9. Terry Gaasterland, Parke Godfrey, and Jack Minker. An overview of cooperative answering. *J. Intell. Inf. Syst.*, 1(2):123–157, 1992.
10. Amir Gilad, Aviram Imber, and Benny Kimelfeld. The consistency of probabilistic databases with independent cells. In *ICDT*, volume 255 of *LIPIcs*, pages 22:1–22:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.
11. Matthew Johnson, Barnaby Martin, Sukanya Pandey, Daniël Paulusma, Siani Smith, and Erik Jan van Leeuwen. Edge Multiway Cut and Node Multiway Cut Are Hard for Planar Subcubic Graphs. In Hans L. Bodlaender, editor, *19th Scandinavian Symposium and Workshops on Algorithm Theory (SWAT 2024)*, volume 294 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 29:1–29:17, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
12. Youri Kaminsky, Benny Kimelfeld, Ester Livshits, Felix Naumann, and David Wajc. Repairing databases over metric spaces with coincidence constraints. In *ICDT*, volume 328 of *LIPIcs*, pages 14:1–14:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.
13. Solmaz Kolahi and Laks V. S. Lakshmanan. On approximating optimum repairs for functional dependency violations. In *ICDT*, volume 361 of *ACM International Conference Proceeding Series*, pages 53–62. ACM, 2009.
14. Nick Koudas, Avishek Saha, Divesh Srivastava, and Suresh Venkatasubramanian. Metric functional dependencies. In *ICDE*, pages 1275–1278. IEEE Computer Society, 2009.
15. Ester Livshits, Benny Kimelfeld, and Sudeepa Roy. Computing optimal repairs for functional dependencies. In *PODS*, pages 225–237. ACM, 2018.
16. Ester Livshits, Benny Kimelfeld, and Sudeepa Roy. Computing optimal repairs for functional dependencies. *ACM Trans. Database Syst.*, 45(1):4:1–4:46, 2020.
17. Andrei Lopatenko and Leopoldo E. Bertossi. Complexity of consistent query answering in databases under cardinality-based and incremental repair semantics. In *ICDT*, pages 179–193, 2007.
18. Dongjing Miao, Pengfei Zhang, Jianzhong Li, Ye Wang, and Zhipeng Cai. Approximation and inapproximability results on computing optimal repairs. *VLDB J.*, 32(1):173–197, 2023.

A Subset Repairs versus Update Repairs

It is instructive to contrast the situation with that of *subset* repairs, which are obtained by deleting a minimum number of tuples rather than by updating cells. For subset repairs, Livshits et al. [16] obtained a *complete* dichotomy: for *every* set of FDs, computing an optimal subset repair is either solvable in polynomial time or NP-hard (in fact, APX-complete). Their hardness argument proceeds in two steps. First, they establish hardness *directly* for a small collection of "core" FD sets, each by a reduction from a well-known NP-hard problem. Second, they lift this hardness to every remaining intractable FD set by means of *fact-wise reductions*: a fact-wise reduction maps each tuple of a relation over one schema to a tuple of a relation over another schema in a way that preserves both consistency and inconsistency. Such a map induces a one-to-one correspondence between the subset repairs of the source relation and those of the target, and hardness therefore transfers immediately from the core sets to all others.

This machinery is unavailable for update repairs, which is a central reason the update-repair picture has remained so much coarser. A fact-wise reduction relates two relations over possibly *different* schemas, and in order to encode the source dependencies it typically represents the value of a single source attribute using *several* attributes of the target relation, so that one source value determines several target values. Two obstacles then arise.

First, a single cell update in the source relation forces an entire *group* of cell updates in the target relation, and the size of this group may differ from one source attribute to another. The number of modified target cells is therefore not a fixed multiple of the number of modified source cells, so the reduction cannot preserve repair cost up to a fixed factor, and the tight, cost-preserving correspondence on which the subset-repair argument relies already fails. Second, and more fundamentally, the correspondence breaks down in the reverse direction. An update repair of the target relation is under no obligation to respect these groups: it may modify the target cells that are determined by a single source cell *inconsistently*, changing some of them but not others, or changing them to values that no longer agree. Such a target repair reflects no update of the source relation at all, since there each cell is atomic and is either left unchanged or updated as a whole. There is thus no faithful way to translate an optimal update repair of the target relation back into one of the source relation of comparable cost, and the fine-grained, cell-level cost accounting that defines an *optimal* update repair is not preserved. For these reasons, hardness cannot simply be transported from a handful of core FD sets to all the rest by means of fact-wise reductions, and different machinery is required.

B Approximation

Beyond these exact results, a substantial body of work targets the optimization problem through approximation and heuristics. Kolahi and Lakshmanan [13] show that, for every fixed set of FDs, an optimal U-repair can be approximated in polynomial time to within some *fixed* constant factor. This factor cannot be pushed arbitrarily close to 1, however: already for some fixed set of FDs, no polynomial-time approximation scheme exists; that is, no polynomial-time $(1+\epsilon)$-approximation for every $\epsilon > 0$, unless P = NP. Livshits et al. [16] revisit the approximability of optimal U-repairs and put forward another constant-factor guarantee, by reducing the problem to computing an optimal subset repair (minimizing tuple deletions), which itself is 2-approximable. More recently, Miao et al. [18] proposed additional approximation algorithms, along with new inapproximability bounds, for computing optimal repairs. A variety of cost-based heuristics and sampling-based methods have also been

proposed for repairing FD violations by value modification [3, 4, 7]. To the best of our knowledge, however, no further *exact* tractability result and no further hardness result have been established since.

C Missing Proofs of Section 4

This section proves Theorem 6: for every $c \geq 3$, computing an optimal U-repair for Δ_c^{sc} is NP-hard. The proof consists of three steps. We first show that R3DM, the restricted variant of three-dimensional matching from which we reduce, is NP-complete (Lemma 15). We then reduce R3DM to our problem for three attributes (Lemma 16). Finally, we reduce the problem for three attributes to the problem for c attributes, for every $c > 3$ (Lemma 17). Throughout, we specify a tuple by listing its values for $A_1, \ldots, A_c$ in this order.

C.1 Hardness of R3DM

We first prove that the restricted matching problem that we reduce from is intractable. Recall that an instance of R3DM consists of three disjoint sets X, Y and Z of size q and a set $S \subseteq X \times Y \times Z$ of triples that satisfies the two conditions LI and TF, and that the goal is to decide whether S has a perfect matching, that is, q pairwise disjoint triples.

▶ **Lemma 15.** R3DM *is NP-complete.*

Proof. Membership in NP is clear, so we prove hardness by a reduction from 3DM. Let (X, Y, Z, S) be an instance of 3DM, where $|X| = |Y| = |Z| = q$ and $|S| = m$. For every triple $s = (x, y, z) \in S$ we introduce six fresh elements, namely a_1^s and a_2^s that we add to X, b_1^s and b_2^s that we add to Y, and c_1^s and c_2^s that we add to Z; these $6m$ elements are distinct from one another and from the elements of $X \cup Y \cup Z$. We call them the *internal* elements of s and the elements of $X \cup Y \cup Z$ the *original* ones. We replace s by the *gadget* G_s that consists of the five triples

$$\sigma_1^s = (x, b_1^s, c_1^s), \quad \sigma_2^s = (a_1^s, y, c_2^s), \quad \sigma_3^s = (a_2^s, b_2^s, z), \quad \sigma_4^s = (a_1^s, b_2^s, c_1^s), \quad \sigma_5^s = (a_2^s, b_1^s, c_2^s).$$

The three sides of the constructed instance are of size $q + 2m$ each, the constructed set $\hat{S}$ of triples is of size $5m$, and a perfect matching consists of $q + 2m$ triples. The construction is clearly in polynomial time.

Next, we show that $\hat{S}$ satisfies LI and TF. We use two properties of the construction. First, every triple of $\hat{S}$ contains at most one original element, since σ_1^s, σ_2^s and σ_3^s contain one each and σ_4^s and σ_5^s contain none. Second, the internal elements of s are introduced for s alone, so they occur only in the triples of G_s.

For LI, consider two distinct triples of $\hat{S}$. If they belong to distinct gadgets, then, by the second property, every element that they share is original, and, by the first, each of them has at most one such element; hence, they share at most one element. If they belong to the same gadget G_s, then a direct inspection of the five triples above shows that they share at most one element; the inspection also shows that they share an element precisely when one of them is σ_1^s, σ_2^s or σ_3^s and the other is σ_4^s or σ_5^s.

For TF, suppose that $\hat{S}$ contains three distinct triples t_1, t_2 and t_3 and three distinct elements u, v and w such that $u, v \in t_1$, $u, w \in t_2$ and $v, w \in t_3$. Every one of the three triples contains two of the three elements, so at most one of u, v and w is original: if two of them were, then some triple would contain both, contradicting the first property. At least two of the elements are therefore internal, say u and v. By the second property, t_1 and t_2

belong to the same gadget, since they share the internal element u, and so do t_1 and t_3, since they share the internal element v; thus, all three belong to a single gadget G_s. This is impossible: by the inspection above, no two of σ_1^s, σ_2^s and σ_3^s share an element and neither do σ_4^s and σ_5^s, so among any three triples of G_s there are two that share no element.

Now, given a perfect matching of S, we construct one of $\hat{S}$. Let $M \subseteq S$ be a perfect matching of the original instance, so $|M| = q$. We select the triples σ_1^s, σ_2^s and σ_3^s for every $s \in M$, and the triples σ_4^s and σ_5^s for every $s \in S \setminus M$. Each of the two selections covers the six internal elements of G_s exactly once, and the first covers, in addition, the three original elements of s. Since M is a perfect matching, every original element is covered exactly once, and the number of selected triples is $3q + 2(m - q) = q + 2m$, as required.

Conversely, let $\hat{M}$ be a perfect matching of $\hat{S}$. We construct a perfect matching of S. Let $s \in S$. As noted above, the six internal elements of s occur only in the triples of G_s, so $\hat{M}$ covers each of them by a triple of G_s; we determine which triples of G_s it uses. If $\sigma_4^s \in \hat{M}$, then a_1^s, b_2^s and c_1^s are covered, and the only triple of G_s that covers a_2^s without reusing them is σ_5^s; the two together cover the six internal elements, and none of s's original elements is covered by G_s. If $\sigma_4^s \notin \hat{M}$, then a_1^s forces $\sigma_2^s \in \hat{M}$, the element b_2^s forces $\sigma_3^s \in \hat{M}$, and c_1^s forces $\sigma_1^s \in \hat{M}$; these three cover the six internal elements, and they cover the original elements of s as well. Hence, every gadget either covers all three original elements of its triple or covers none of them. As every original element is covered exactly once, the set M of the triples s whose gadget covers them is a perfect matching of the original instance. ◀

C.2 Proof of Theorem 6

We first prove the statement for three attributes.

▶ **Lemma 16.** *Computing an optimal U-repair for Δ_3^{sc} is NP-hard.*

Proof. We reduce from R3DM, which is NP-complete by Lemma 15. Let (X, Y, Z, S) be an instance of R3DM, where $|X| = |Y| = |Z| = q$ and $|S| = m$, and where we may assume that $m \geq q \geq 1$, since otherwise S has no perfect matching. We refer to the members of $X \cup Y \cup Z$ as *S-elements*. We construct a relation r over A_1, A_2 and A_3 and a budget K such that S has a perfect matching if and only if r has a consistent update r' with $\mathit{dist}(r', r) \leq K$. For $c = 3$, the target tuple of a cluster is a triple, and we call it the *target triple* of the cluster; recall that the target triples of distinct clusters disagree on A_1, A_2 and A_3.

We set:

$$K \;:=\; 8m - 5q \qquad \text{and} \qquad N \;:=\; K + 1.$$

For every triple $s = (x, y, z) \in S$, the relation r contains the four *gadget tuples*

$$t_s = (x, y, z) \qquad u_s^1 = (c_1^s, y, z) \qquad u_s^2 = (x, c_2^s, z) \qquad u_s^3 = (x, y, c_3^s)$$

where c_1^s, c_2^s and c_3^s are *fresh constants*. We refer to t_s as an *S-tuple*, and to u_s^1, u_s^2 and u_s^3 its *auxiliary tuples*; note that u_s^j contains c_j^s in A_j and agrees with t_s on the other two attributes. In addition, r contains, for every $j \in \{1, 2, 3\}$, an *enforcing group*: a set of N copies of a tuple that contains c_j^s in A_j and, in the two other attributes, the values $d_{j,i}^s$; that is, N copies of each of the following three tuples:

$$(c_1^s, d_{1,2}^s, d_{1,3}^s) \qquad (d_{2,1}^s, c_2^s, d_{2,3}^s) \qquad (d_{3,1}^s, d_{3,2}^s, c_3^s)$$

The $3m$ fresh constants c_j^s and the $6m$ values $d_{j,i}^s$ are distinct from one another and from the elements. Figure 1 depicts the tuples introduced for a single triple s, with the superscript

s omitted. Altogether, r consists of the $4m$ gadget tuples and of the $3mN$ tuples of the enforcing groups, and it is constructed in polynomial time.

Now, given a perfect matching of S, we construct a consistent update of r. Let $M \subseteq S$ be a perfect matching, and recall that $|M| = q$. We leave every enforcing group unchanged, so that each of its tuples is its own target triple.

- For $s = (x, y, z) \in M$, we assign the target triple (x, y, z) to the four gadget tuples of s. The S-tuple t_s is then left unchanged, and each auxiliary tuple u^j_s changes the single cell that contains c^s_j, at a total cost of 3 for each of the q triples in M.
- Now let $s' = (x', y', z') \in S \setminus M$. As M covers every S-element, it contains a triple $s_{x'} = (x', y_{x'}, z_{x'})$ that includes x', and it contains a triple $s_{y'} = (x_{y'}, y', z_{y'})$ that includes y'. Both tuples are distinct from s', so, by LI, each of them shares with s' only one element (x' or y'), and hence $x_{y'} \neq x'$, $y_{x'} \neq y'$, $z_{x'} \neq z'$ and $z_{y'} \neq z'$. We assign:
  - The target triple $s_{x'}$ to $t_{s'}$, $u^2_{s'}$ and $u^3_{s'}$;
  - The target triple $s_{y'}$ to $u^1_{s'}$.

  Since no fresh constant is an S-element, each of $t_{s'}$, $u^2_{s'}$ and u^3_s keeps only its value x' in A_1, and $u^1_{s'}$ keeps only its value y' in A_2. Hence, each of the four tuples in the gadget changes two cells, at a total cost of 8. Hence, we get a total cost of 8 for each of the $m - q$ triples s' in $S \setminus M$.

In conclusion, the resulting update r' satisfies $dist(r', r) = 3q + 8(m - q) = 8m - 5q = K$, as required. It remains to verify that r' is consistent, that is, that the target triples of distinct clusters disagree on A_1, A_2 and A_3. These target triples are the q triples of M and the $3m$ tuples of the enforcing groups. Two triples of M are disjoint and therefore disagree on every attribute; a triple of M contains elements only, whereas a tuple of an enforcing group contains no elements; and two tuples of distinct enforcing groups disagree on every attribute, since the values c^s_j and $d^s_{j,i}$ are all distinct.

Next, given a consistent update of r, we construct a perfect matching of S. Let r' be a consistent update of r with $dist(r', r) \leq K$. Then r' includes several clusters, each consisting of copies of a target tuple. We will show that at least q of the target tuples must be S-tuples. From the consistency of r' we can then conclude that these q S-tuples form a perfect matching M, as required. We begin with two observations.

(O1) *We can assume that every enforcing group is unchanged in r'.* Since $N > K$, at least one tuple from the enforcing group remains unchanged, so if any tuple in this group is changed, we can reverse its changes without causing any violation.

(O2) *We can further assume that every gadget tuple keeps at least one of its three cells.* Let $t \in r$ be a gadget tuple, and suppose that all three values of t are updated in r'. If any target tuple $t' \in r'$ coincides with t on at least one attribute, then we can update t to t' and reduce the cost by one. If no such t' agrees, then we can again reverse the updates of t' and reduce the cost.

The cost of r' is then the number of cells changed within the gadget tuples. By (O2), we can assume that the cost c_t incurred by each individual gadget tuple t is at most 2. For convenience, in the remainder of this proof, we refer to $2 - c_t$ as the *saving* of t. We further refer to the *saving of a cluster* as the total savings of all tuples in the cluster. With zero savings, we have $4m$ tuples with a total cost of $8m$. Since the total cost of r' is at most $K = 8m - 5q$, we conclude that the total savings over all clusters are at least $5q$.

Next, we analyze the savings of clusters by the type of their target tuple. Before that, we need another observation.

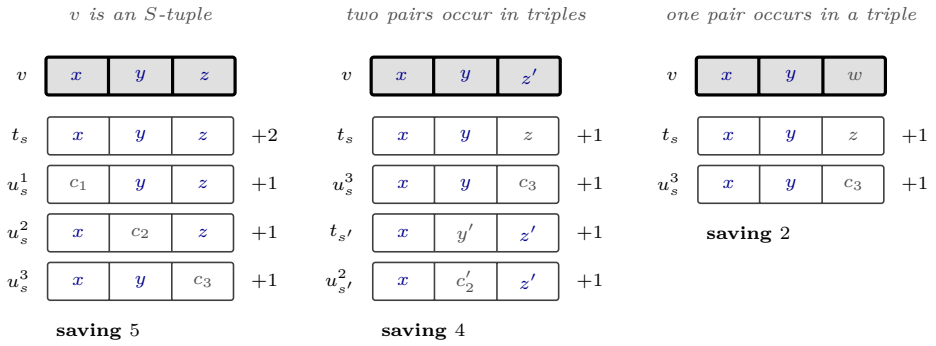

Figure 3 The three savings that a cluster can attain, with the target triple v on top and, below it, the gadget tuples that save through it. Cells that agree with v are shown in blue and cells that differ from it in gray, and the saving of each tuple is written on its right. On the left, all three pairs of elements of v occur in the triple s; in the middle, the triples $s = (x, y, z)$ and $s' = (x, y', z')$ share only the element x, and v contains a pair of each; on the right, w is either not an element or an element that occurs in no triple together with x or with y.

(O3) *We can assume that whenever a target tuple is not equal to an enforcing tuple, then each value is either an S-element or a new value that does not occur in r.* This means that values of the form $d^s_{j,i}$ and c^s_j cannot be used in clusters where the target tuple is not equal to an enforcing tuple. This follows directly from (O1), since enforcing tuples from r remain intact in r'; hence, if any $d^s_{j,i}$ or c^s_j is retained in a gadget tuple, then the tuple needs to agree on all attributes with the enforcing tuple that contains $d^s_{j,i}$ or c^s_j.

We consider five types of clusters, each defined by a property of its target cluster v. For each cluster, we compute a bound on its savings.

(a) *No two elements of v co-occur together inside a triple of S.* Due to the construction of the gadget tuples, and due to (O3), every gadget tuple differs from v in at least two attributes. Hence, this cluster saves nothing.

(b) *Exactly one pair of values in v co-occur together in a triple of S.* Consider the two S-elements of v that co-occur together in S. By LI, exactly one S-triple contains both of them. By construction, precisely two of its gadget tuples contain both—its S-tuple and one auxiliary tuple; if assigned to the cluster, each of them keeps two cells and saves one. By (O3), every other tuple in the cluster differs from v in at least two attributes, so it saves nothing. So, this cluster saves at most $2 = 1 + 1$.

(c) *Exactly two distinct pairs of values in v co-occur inside triples of S.* Due to TF, the third pair of values *does not* co-occur inside any triple of S. Hence, this case is similar to (b), but with twice the bound. The cluster therefore saves at most $4 = 2 + 2$.

(d) *v is an S-tuple.* Let $s = (x, y, z) \in S$ be this tuple. If assigned to this cluster, the tuple t_s keeps all three of its cells and saves 2, and each auxiliary tuple u^j_s keeps the two cells that it inherits from t_s and saves 1. Every other gadget tuple belongs to a triple $s' \neq s$, which, by LI, shares at most one element with s; hence, assigning it to the cluster saves nothing. The cluster saves at most $5 = 2 + 1 + 1 + 1$.

Note that cases (a)-(d) are pairwise disjoint. Also, due to the TF, they cover all possible clusters. Let n_x be the number of distinct target tuples of type (x). To complete the proof, we will show that $n_d = q$; hence, the the target tuples of type (d) form a perfect matching.

Let T be the total savings. The case analysis shows that $0n_a + 2n_b + 4n_c + 5n_d \geq T$. On the other hand, we know that $T \geq 5q$. Hence,

$$2n_b + 4n_c + 5n_d \geq 5q\,. \tag{1}$$

Let N be the number of S-elements that occur in the target triples. Then $3q \geq N$, which is the total number of S-elements. Two distinct clusters disagree on every attribute, so no S-element is contained in more than one target tuple. Each target tuple of type (b) includes at least two S-elements, and each target tuple of types (c) and (d) includes at least three S-elements. We conclude that $N \geq 2n_b + 3n_c + 3n_d$. Hence,

$$3q \geq 2n_b + 3n_c + 3n_d. \tag{2}$$

Combining Equation (1) and Equation (2), we get that

$$3q \geq 2n_b + 3n_c + 3n_d = 2n_b + 4n_c + 5n_d - (n_c + 2n_d) \geq 5q - (n_c + 2n_d)$$

Hence, $2q \leq n_c + 2n_d \leq 2n_c + 2n_d$, and therefore $n_c + n_d \geq q$. Using Equation (2), we conclude that $3q \geq 2n_b + 3n_c + 3n_d \geq 2n_b + 3q$, hence $n_b = 0$. With that, from Equation (2) we conclude that $3q \geq 3n_c + 3n_d$, hence $n_c + n_d \leq q$. Thus, $n_c + n_d = q$. Finally, recalling again Equation (1), we conclude that

$$5q \leq 4n_c + 5n_d = 4n_c + 4n_d + n_d = 4q + n_d$$

and, therefore, $n_d \geq q$ (actually, $n_d = q$ and $n_c = 0$), as claimed. ◀

It remains to pass from three attributes to any larger number, which we do by padding every tuple with values that no consistent update within the budget can preserve.

▶ **Lemma 17.** *Let $c > 3$. Given a relation r over A_1, A_2 and A_3 with n tuples and a budget K, one can construct in polynomial time a relation r^* over $A_1, \ldots, A_c$ such that r has a consistent update of cost at most K with respect to Δ_3^{sc} if and only if r^* has a consistent update of cost at most $K + n(c-3)$ with respect to Δ_c^{sc}.*

Proof. Let $N := K + n(c-3) + 1$. For every tuple t of r, the relation r^* contains the tuple t^* that agrees with t on A_1, A_2 and A_3 and contains a *private* value in each of $A_4, \ldots, A_c$. In addition, for every private value p, say the one that t^* contains in A_j, the relation r^* contains an *enforcing group* of N copies of the *enforcing tuple* e_p that contains p in A_j and, in every other attribute, a value that occurs nowhere else. All of these values are distinct and occur in no tuple of r.

Now, given a consistent update of r, we construct a consistent update of r^*. Let r' be a consistent update of r with $dist(r', r) \leq K$. We leave every enforcing group unchanged and assign to the tuples t^* the target triples of r', each extended by values that occur nowhere in r^* and are distinct across clusters. Every t^* then changes its $c-3$ private cells in addition to the cells that t changes in r', at a total cost of at most $K + n(c-3)$. The result is consistent: the extended target tuples pairwise disagree on A_1, A_2 and A_3, as do the target triples of r', and they agree with no enforcing tuple in any attribute.

Next, given a consistent update of r^*, we construct a consistent update of r. Let $\hat{r}$ be a consistent update of r^* with $dist(\hat{r}, r^*) \leq K + n(c-3)$. Since N exceeds this budget, every enforcing group retains an unchanged copy, and so every enforcing tuple e_p is a target tuple of $\hat{r}$. As the target tuples of distinct clusters disagree on every attribute, we conclude the following.

($\star$) For every private value p, the only target tuple of $\hat{r}$ that contains p in the attribute in which e_p contains it is e_p itself.

We say that a tuple t^* is *captured* if its target tuple is an enforcing tuple e_p. Since e_p contains, in every attribute other than the one that holds p, a value that occurs nowhere else, a captured tuple retains at most one cell and therefore costs at least $c-1$.

We first show that we may assume that no tuple of $\hat{r}$ is captured. Let t^* be a captured tuple. If some target tuple v of $\hat{r}$ that is not an enforcing tuple agrees with t^* on an attribute, then we move t^* to the cluster of v; the tuple t^* then retains a cell and costs at most $c-1$. Otherwise, we place t^* in a cluster of its own, the target tuple of which agrees with t^* on A_1, A_2 and A_3 and contains, in each of $A_4, \ldots, A_c$, a value that occurs nowhere in r^* and in no other target tuple; the tuple t^* then costs $c-3$. In both cases, the result is again a consistent update of r^*, and its cost is no higher than that of $\hat{r}$. This is clear in the first case. In the second case, the new target tuple contains new values in $A_4, \ldots, A_c$ and agrees with t^* on A_1, A_2 and A_3, so it suffices to observe that no other target tuple agrees with t^* on A_1, A_2 or A_3. For a target tuple that is not an enforcing tuple, this is the assumption of the case; and an enforcing tuple contains in A_1, A_2 and A_3 values that occur nowhere else in r^*, whereas t^* contains there values of r. Each application of this transformation reduces the number of captured tuples by one, so we may indeed assume that $\hat{r}$ has none.

By ($\star$), a tuple t^* that is not captured retains none of its private cells, and hence it changes all $c-3$ of them, in addition to the cells among A_1, A_2 and A_3 on which it differs from its target tuple. Denoting by d_t the number of the latter, we obtain that $dist(\hat{r}, r^*) \geq n(c-3) + \sum_{t \in r} d_t$ and, therefore, that $\sum_{t \in r} d_t \leq K$.

We construct r' by assigning to every tuple t of r the restriction of the target tuple of t^* to A_1, A_2 and A_3. Every tuple then changes precisely d_t cells, so $dist(r', r) = \sum_{t \in r} d_t \leq K$. Finally, r' is consistent: two tuples of r' that originate in the same cluster of $\hat{r}$ are equal, and two tuples that originate in distinct clusters disagree on each of A_1, A_2 and A_3, since the target tuples of distinct clusters disagree on every attribute. ◀

THEOREM 6. *For every $c \geq 3$, computing an optimal U-repair for Δ_c^{sc} is NP-hard.*

Proof. For $c = 3$, this is Lemma 16, the proof of which shows that it is NP-complete to decide whether a relation over A_1, A_2 and A_3 has a consistent update of cost at most a given budget. For $c > 3$, Lemma 17 reduces this decision problem, in polynomial time, to the corresponding problem for Δ_c^{sc}. ◀

D Missing Proofs of Section 5

In this section, we provide the missing proofs for Section 5.

D.1 Proof of Proposition 8

PROPOSITION 8. *Computing an optimal U-repair for $A \leftrightarrow B \rightarrow C$ is NP-hard.*

Proof. We reduce from VERTEX COVER. Let $G = (V, E)$ be a graph and let k be a bound on the size of a vertex cover; we may assume that $k \leq |V|$, since V itself is a vertex cover. We set $L := |V| + 1$ and $M := L|E| + |V| + 1$.

We construct a relation r over $\{A, B, C\}$, writing a tuple as a triple (a, b, c) that lists its values for A, B and C. The relation consists of the following tuples.

- *Anchors.* For every vertex $v \in V$, the relation contains the *anchors* of v, namely $M+1$ copies of $(v, v, 0)$ and M copies of $(v, v, 1)$.
- *Edge gadgets.* For every edge $\{u, v\} \in E$, the relation contains the *edge gadget* of $\{u, v\}$, namely L copies of $(u, v, 1)$, where the two endpoints are ordered arbitrarily.

The budget is $K_r := M|V| + L|E| + k$. We show that G has a vertex cover of size at most k if and only if r has a consistent update r' with $\mathit{dist}(r', r) \leq K_r$.

Given a vertex cover of G, we construct a consistent update of r. Let S be a vertex cover of G with $|S| \leq k$, and let $c_v := 1$ if $v \in S$ and $c_v := 0$ otherwise. We change to c_v the C-value of every anchor of v whose C-value differs from c_v, at a cost of $M+1$ if $v \in S$ and of M otherwise, and therefore at a total cost of $M|V| + |S|$. For every edge $\{u, v\}$, we pick an endpoint in S, say u, and change to u the B-value of the L copies of $(u, v, 1)$, at a cost of L per edge. The total cost is $M|V| + |S| + L|E| \leq K_r$.

Every tuple of the resulting relation is of the form (v, v, c_v) for a vertex v: the anchors of v by construction, and each edge tuple because $c_u = 1$ for the chosen endpoint u. The FDs $A \to B$ and $B \to A$ hold because the A- and B-values of every tuple coincide, and $B \to C$ holds because all the tuples with $B = v$ have the C-value c_v.

Next, given a consistent update of r, we construct a vertex cover of G. Let r' be a consistent update of r with $\mathit{dist}(r', r) \leq K_r$. Since $r' \models A \leftrightarrow B$, the pairs of A- and B-values of r' form a partial bijection: all the tuples that share their A-value also share their B-value, and vice versa. Moreover, since $r' \models B \to C$, all the tuples that share their B-value share their C-value as well.

Consider the anchors of a vertex v. In r, all of them use the pair (v, v) in A and B, and they split into a group of $M+1$ tuples with $C = 0$ and a group of M tuples with $C = 1$, violating $B \to C$. Hence, we must modify at least one cell in every tuple of one of the two groups, at a cost of at least M per vertex. The budget is $M|V| + L|E| + k$, so this leaves a slack of $L|E| + k$.

Pulling the two groups apart is expensive. We say that an anchor of v *stays* if it retains both its A-value v and its B-value v in r', and that it *leaves* otherwise. Suppose that at least one anchor of v stays and at least one leaves. The anchors that stay use the pair (v, v), so every tuple of r' with $A = v$ has $B = v$ and every tuple of r' with $B = v$ has $A = v$. Therefore, an anchor that leaves cannot retain just one of $A = v$ and $B = v$: changing one of the two alone collides with the group that stays. It must change both, at a cost of 2 per leaving tuple. As the smaller group has M tuples, pulling a group apart costs at least $2M$. If no anchor of v stays, then each of the $2M+1$ anchors of v changes a cell, which costs at least $2M+1$. As M exceeds the slack, neither option is affordable. Hence, every vertex has an anchor that stays, and every vertex unifies its anchors on a single value c_v of C: the anchors that stay share the B-value v, hence they share their C-value as well, and we denote it by c_v. An anchor that stays costs at least 1 if its C-value in r differs from c_v, so the anchors of v cost at least M if $c_v = 0$, at least $M+1$ if $c_v = 1$, and at least $2M+1$ if $c_v \notin \{0, 1\}$. The last option exceeds the slack, so $c_v \in \{0, 1\}$. Let $S := \{v \in V \mid c_v = 1\}$; the anchors cost at least $M|V| + |S|$.

The edge gadgets test whether S covers E. Since an anchor of v stays and carries the pair (v, v), the FD $A \to B$ implies that every tuple of r' with $A = v$ has $B = v$, and $B \to C$ implies that it has $C = c_v$; symmetrically, every tuple with $B = v$ has $A = v$ and $C = c_v$. Now consider the gadget of an edge $\{u, v\}$. None of its L copies of $(u, v, 1)$ can retain both $A = u$ and $B = v$, so each of them must join the cluster of u or the cluster of v, or leave both. Joining that of u amounts to changing the B-value to u, and it changes nothing else exactly when $c_u = 1$, since then $C = 1$ is already the C-value of the cluster; if $c_u = 0$, the copy must

also change its C-value to 0. The argument for joining the cluster of v is symmetric. A copy that joins neither cluster changes both its A-value and its B-value. Hence, the gadget costs at least L, and at least $2L$ if $c_u = c_v = 0$, that is, if the edge is not covered by S.

Let E_0 be the set of the edges that S does not cover. Putting the bounds together,

$$K_r \;\geq\; dist(r', r) \;\geq\; M|V| + |S| + L|E| + L|E_0|, \qquad \text{hence} \qquad |S| + L|E_0| \;\leq\; k.$$

Since $L = |V| + 1 > k$, we get $E_0 = \emptyset$, so S is a vertex cover, and $|S| \leq k$. ◀

D.2 Proof of Proposition 9

PROPOSITION 9. *Computing an optimal U-repair for $A \to B \leftrightarrow C$ is NP-hard.*

Proof. We reduce from 3SAT, assuming that every clause consists of three literals over three distinct variables. Let φ be a formula with variable set V and m clauses, and set $L := 4m+1$. With every variable v we associate two fresh values T_v and F_v, one for each truth value. The constructed relation over $\{A, B, C\}$ consists of:

- a *variable gadget* for every variable v, namely L copies of (v, v, T_v) and L copies of (v, v, F_v), and
- a *clause gadget* for every clause $c = (\ell_1 \vee \ell_2 \vee \ell_3)$ over the variables v_1, v_2, v_3, namely the three tuples:

$$(c, v_1, w_3) \qquad (c, v_2, w_1) \qquad (c, v_3, w_2)$$

  where w_i is the value of v_i that satisfies ℓ_i, that is, T_{v_i} if ℓ_i is positive and F_{v_i} if ℓ_i is negative.

Note that the indices in a clause gadget are shifted: v_1, v_2 and v_3 coincide with w_3, w_1 and w_2, respectively. For example, if c is $x \vee y \vee \neg z$, then the three tuples are:

$$(c, x, F_z) \qquad (c, y, T_x) \qquad (c, z, T_y)$$

The budget is $K := L|V| + 4m$.

Given a satisfying assignment τ of φ, we construct a consistent update of cost exactly K. Denote by $\sigma(v)$ the value T_v if $\tau(v) = \mathbf{true}$ and the value F_v otherwise.

- For every variable v, we change to $\sigma(v)$ the C-value of the L tuples of its gadget that carry the other value (cost $L|V|$ in total).
- For every clause c, we pick a literal ℓ_i that τ satisfies, and we replace the three tuples of its gadget by (c, v_i, w_i); one of the three has already the B-value v_i, another already has the C-value $w_i = \sigma(v_i)$, and the third has neither (cost $1 + 1 + 2 = 4$ per clause, hence $4m$ in total).

The resulting relation r' is consistent: it satisfies $A \to B$ since the tuples that share an A-value are those of a single gadget, and it satisfies $B \leftrightarrow C$ since its pairs of B- and C-values are precisely the pairs $(v, \sigma(v))$.

Now suppose that we are given a consistent update r' of cost at most K. Since $B \leftrightarrow C$, the pairs of B-values and C-values that occur in it form a partial bijection, which we denote by σ. If no tuple has the B-value v, then $\sigma(v)$ is undefined, and we let it be a value that is neither T_v nor F_v, say $\diamond$. Hence, $\sigma(v)$ is defined for every variable v.

Consider the gadget of a variable v, and let t be a tuple of the gadget whose C-value in r differs from $\sigma(v)$. The update cannot keep both the B-value and the C-value of t, and it therefore changes at least one cell of t. Thus, the gadget costs at least the number of its tuples whose C-value differs from $\sigma(v)$. As the gadget splits evenly between T_v and F_v, this number is at least L if $\sigma(v)$ is one of T_v and F_v, and at least $2L$ if it is a different C-value in r'. The cost is also at least $2L$ if $\sigma(v) = \diamond$, since it means that all $2L$ occurrences of v had to disappear in the update.

In conclusion, the variable gadgets cost at least $L|V|$ in total, and at least $L|V| + L$ if σ maps some variable to neither of its two values. Since L exceeds the remaining budget of $4m$, the latter is impossible. Hence, σ maps every variable v to one of T_v and F_v, and we can treat it as a truth assignment τ. We complete the proof by showing that τ is a satisfying assignment. Since the remaining budget is $4m$, it suffices to show that a clause gadget costs at least 4, and that it costs exactly 4 only if τ satisfies its clause. We do so next.

Due to the shift, every tuple of a clause gadget pairs a variable with a value of a *different* variable; no such pair belongs to σ, so each of the three tuples changes its B-value or its C-value, and the gadget costs at least 3. If the three tuples do not all retain the A-value c, then at least one of them changes its A-value as well, and the gadget costs at least 4. If they all retain it, then A determines both B and C, so the three tuples collapse into a single pair of σ; their three B-values are distinct and so are their three C-values, hence this pair agrees with at most two of the six B-cell and C-cells of the gadget, and the gadget again costs at least 4. In both cases, a cost of exactly 4 forces a pair of $(v, \sigma(v))$ that agrees with one B-cell and one C-cell of the gadget:

- In the first case, exactly one tuple changes its A-value and one of its B of C values, so the two tuples that retain their A have a budget of 2. Since they agree on A, they should be updated into identical tuples, and the only way we can do so is by retaining at least one of the w_i of the gadget, and w_i satisfies the clause (by construction).
- In the second case, where A-values are retained, the three tuples should become identical, and the gadget should update at least two B-cells and at least two C-cells. Hence, it updates precisely two C-cells, and again at least one w_i is retained and satisfies the clause.

We conclude that the clause is necessarily satisfied and τ is a satisfying assignment, as claimed. ◀

D.3 Proof of Proposition 10

Proposition 10. *Computing an optimal U-repair for $A \leftrightarrow B \rightarrow C \leftrightarrow D$ is NP-hard.*

Proof. We reduce from $A \leftrightarrow B \rightarrow C$, which is intractable by Proposition 8 and consists of the first three attributes of $A \leftrightarrow B \rightarrow C \leftrightarrow D$. In the spirit of Lemma 17, the reduction pads every tuple with a private value in the new attribute D and anchors this value by an enforcing group.

Let r be a relation over $\{A, B, C\}$ with n tuples, let K be a budget, and set $N := K+n+1$. For every $i \in \mathsf{tids}(r)$, the relation r' over $\{A, B, C, D\}$ contains:

- the *main tuple* M_i that agrees with $r[i]$ on A, B and C and contains a *private* value d_i in D, and
- for every private value d_i, the relation r' contains an *enforcing group* of N copies of the *enforcing tuple* E_i that contains d_i in D and, in each of A, B and C, a value that occurs nowhere else.

The budget is $K + n$. We show that r has a consistent update of cost at most K with respect to $A \leftrightarrow B \rightarrow C$ if and only if r' has a consistent update of cost at most $K + n$ with respect to $A \leftrightarrow B \rightarrow C \leftrightarrow D$.

Now, given a consistent update of r, we construct a consistent update of r', Let s be a consistent update of r with $dist(s, r) \leq K$. We may assume that s uses none of the values that the construction introduces, since renaming the values of s that do not occur in r affects neither its cost nor its consistency. With every value c that occurs in the C-column of s we associate a fresh value δ_c, and we define the update s' of r' that leaves every enforcing group unchanged and replaces M_i by the tuple $(s[i][A], s[i][B], s[i][C], \delta_{s[i][C]})$. Every main tuple changes its D-cell, since $\delta_{s[i][C]}$ is fresh and d_i is not, and the main tuples change $dist(s, r)$ cells in A, B and C in total; hence $dist(s', r') = dist(s, r) + n \leq K + n$.

We verify that $s' \models A \leftrightarrow B \rightarrow C \leftrightarrow D$. Among the main tuples, $A \leftrightarrow B$ and $B \rightarrow C$ hold because $s \models A \leftrightarrow B \rightarrow C$, and $C \leftrightarrow D$ holds because $c \mapsto \delta_c$ is injective. An enforcing tuple E_i shares no value with any other tuple of s' in any attribute: its A-, B- and C-values occur only in its own enforcing group, and its private value d_i is no longer used by M_i. Hence, no FD is violated.

Next, given a consistent update of r', we construct a consistent update of r. Let s' be a consistent update of r' with $dist(s', r') \leq K + n$. Since N exceeds this budget, every enforcing group retains an unchanged copy, and so every enforcing tuple E_i occurs in s'. As the enforcing tuples occur in s', the FDs $C \rightarrow D$ and $D \rightarrow C$ imply that a main tuple agrees with an enforcing tuple on C if and only if it agrees with it on D.

We first show that we may assume that no main tuple agrees with an enforcing tuple on C. Consider a group Q of main tuples that have been updated to the same (a, b, c, d), where (c, d) is the pair of C- and D-values of an enforcing tuple E. Every tuple of Q has changed its C-cell, since c occurs nowhere in r. Moreover, at most one tuple of Q retains its original D-value, namely the main tuple whose private value is d, if it belongs to Q. Let $j \in Q$ and let c' be the original C-value of j. If c' already occurs in s', let d' be the D-value already paired with it; otherwise, let d' be a fresh value. We replace the pair (c, d) of every tuple of Q by (c', d'). If (a, b) is the pair of A- and B-values of E, those cells were already changed, since the values of E occur nowhere in r, and we replace them by a fresh pair as well, so as not to violate $B \rightarrow C$ against E. One C-cell is then restored, namely that of j, and at most one extra D-cell is changed, namely if some tuple of Q had retained d and d' differs from d. Hence, the cost does not increase.

The result is again a consistent update of r'. The tuples of Q now agree on (c', d'). If we replaced A and B, they agree on a fresh pair that occurs nowhere else. If we kept (a, b), then no tuple outside Q has the B-value b: the enforcing tuple E has a different B-value, and every other tuple with $B = b$ would have had $C = c$ by $B \rightarrow C$, hence would have belonged to Q. The value c' occurs in r and is therefore the C-value of no enforcing tuple, so the tuples of Q no longer agree with an enforcing tuple on C or on D. Repeating the modification for every such group, we obtain a consistent update of r' of cost at most $K + n$ in which no main tuple agrees with an enforcing tuple on D, and we assume from now on that s' is such an update.

In s', no main tuple retains its private value, since it would then agree with its own enforcing tuple on D and hence on C. The main tuples therefore change n cells in the attribute D, and so they change at most K cells in A, B and C. Let s be the update of r obtained by restricting the main tuples of s' to A, B and C; then $dist(s, r) \leq K$, and s is consistent, since the FDs $A \rightarrow B$, $B \rightarrow A$ and $B \rightarrow C$ are implied by $A \leftrightarrow B \rightarrow C \leftrightarrow D$ and hence hold in s', in particular among the main tuples. ◀